# Crucible — API Architecture

> *Closes the four gaps identified in CEO review of ARCHITECTURE.md and ARCHITECTURE-FORGE.md. Gap 2: Monte Carlo / Ensemble Runs Gap 3: Output Layer (NarrativeAgent, reports) Gap 4: Persistence (SQLite/Postgres + Redis) Gap 5: [0,1] Normalization Constraint*
>
> *Also specifies: Data Feed Agent (calibration loop), SimSpec versioning, full API surface. Implementation begins Week 3.*

---

## Full System Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CRUCIBLE PLATFORM                             │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  FORGE (intake layer)                                        │   │
│  │                                                              │   │
│  │  ForgeSession ──► ScopingAgent ──► ResearchPipeline          │   │
│  │       │          (Claude API)     (arXiv / SSRN / FRED /     │   │
│  │       │                           World Bank / News)         │   │
│  │       │                                                      │   │
│  │       └──► TheoryMapper ──► SpecBuilder ──► SimSpec (v1)     │   │
│  └──────────────────────────────────┬───────────────────────────┘   │
│                                     │ SimSpec                       │
│  ┌──────────────────────────────────▼───────────────────────────┐   │
│  │  ENGINE LAYER                                                │   │
│  │                                                              │   │
│  │  SimRunner ◄──────────────────────────── EnsembleRunner      │   │
│  │    │  tick loop                           N × SimRunner      │   │
│  │    │  (asyncio.to_thread)                 asyncio.gather     │   │
│  │    │                                      seed perturbation  │   │
│  │    ├──► NarrativeAgent ◄── Claude API                         │   │
│  │    │    (threshold events → plain-English commentary)        │   │
│  │    │                                                         │   │
│  │    └──► SimEvent bus (in-process pub/sub)                    │   │
│  └──────────────────────────────────┬───────────────────────────┘   │
│                                     │                               │
│  ┌──────────────────────────────────▼───────────────────────────┐   │
│  │  PERSISTENCE LAYER                                           │   │
│  │                                                              │   │
│  │  Redis                        Postgres (SQLite in dev)       │   │
│  │  ├── ForgeSession (TTL 24h)   ├── simulations                │   │
│  │  └── live WS state            ├── sim_specs + versions       │   │
│  │                               ├── snapshots                  │   │
│  │                               ├── metric_records             │   │
│  │                               ├── narrative_entries          │   │
│  │                               └── forge_sessions (completed) │   │
│  └──────────────────────────────────┬───────────────────────────┘   │
```

```
│                                    │                                │
│  ┌─────────────────────────────────▼────────────────────────────┐  │
│  │ API LAYER (FastAPI)                                           │  │
│  │                                                               │  │
│  │ POST /simulations          POST /simulations/{id}/ensemble    │  │
│  │ GET  /simulations/{id}     GET  /ensembles/{id}               │  │
│  │ WS   /simulations/{id}/stream                                 │  │
│  │ POST /simulations/{id}/reports                                │  │
│  │ GET  /simulations/{id}/narrative                              │  │
│  │ GET  /simulations/{id}/calibration/proposals                  │  │
│  │ GET  /simspecs/{id}/versions                                  │  │
│  └───────────────────────────────────┬───────────────────────────┘  │
│                                    │                                │
│  ┌────────────────┐    ┌────────────▼──────────────────────────┐  │
│  │ DATA FEED AGENT │    │ CLIENT LAYERS                         │  │
│  │ APScheduler     │    │ Forge UI  (internal consultants)      │  │
│  │ every 6h/sim    │    │ Portal    (client-facing SaaS)        │  │
│  │ CalibrationProposal→ │                                       │  │
│  │ consultant queue│    └───────────────────────────────────────┘  │
│  └────────────────┘                                                 │
└──────────────────────────────────────────────────────────────────────┘
```

---

## Gap 5: `core/spec.py` — `EnvKeySpec` and SimSpec extension

The normalization fix is the lowest-risk change and touches only the spec layer. SimRunner continues computing in `[0, 1]`. The API layer denormalizes before serving.

```python
# core/spec.py (additions)

class EnvKeySpec(BaseModel):
    """
    Display metadata for a normalized environment key.
    SimRunner operates on the normalized float [0,1].
    API and NarrativeAgent use display_value = normalized * scale.
    """
    key:          str   # matches a key in initial_environment
    scale:        float = 1.0    # multiply normalized to get display value
    unit:         str   = ""     # "USD", "billion USD", "% of GDP", "index", "bbl"
    display_name: str   = ""     # human-readable label: "Iranian Military Readiness"
    # optional: log-scale display (e.g. for GDP values)
    log_scale:    bool  = False


class SimSpec(BaseModel):
    # ... (all existing fields unchanged) ...

    # Gap 5 addition: display annotations for env keys
    env_key_specs: list[EnvKeySpec] = Field(default_factory=list)

    # populated by helper at serve time — never persisted
    def display_env(self, normalized: dict[str, float]) -> dict[str, Any]:
        """
        Convert normalized env dict to display values.
        Keys without an EnvKeySpec are returned as-is.
        """
        index = {s.key: s for s in self.env_key_specs}
        out: dict[str, Any] = {}
```

```
    for k, v in normalized.items():
        spec = index.get(k)
        if spec:
            out[k] = {
                "normalized": v,
                "display": v * spec.scale,
                "unit": spec.unit,
                "display_name": spec.display_name or k,
            }
        else:
            out[k] = {"normalized": v, "display": v, "unit": "", "display_name": k}
    return out
```

### Key design decisions

| Decision | Rationale |
|---|---|
| SimRunner stays in `[0,1]` | No numerical instability risk. All existing theories unchanged. |
| `display_env()` on SimSpec, not SimRunner | API layer calls it; engine never sees display values. Clean separation. |
| `EnvKeySpec` is optional per key | Most keys are fine as raw floats. Only add annotation where scale matters for reports. |
| `log_scale` flag | GDP differences are better shown logarithmically. Renderer uses this hint. |

---

## Gap 4: Persistence Layer

### `api/db/schema.py` — Database schema

```
# api/db/schema.py
# SQLAlchemy Core — no ORM, explicit table definitions.
# Dev: SQLite. Prod: Postgres (swap DB_URL env var).

from __future__ import annotations

from sqlalchemy import (
    Column, DateTime, Float, ForeignKey, Index, Integer,
    MetaData, String, Table, Text, func,
)

metadata = MetaData()


# ── sim_specs ──────────────────────────────────────────────────────
# One row per spec version. parent_spec_id creates the version DAG.

sim_specs = Table(
    "sim_specs", metadata,
    Column("spec_id",        String,   primary_key=True),
    Column("name",           String,   nullable=False),
    Column("domain",         String,   nullable=False, default=""),
    Column("version_number", Integer,  nullable=False, default=1),
    Column("parent_spec_id", String,   ForeignKey("sim_specs.spec_id"), nullable=True),
    Column("spec_json",      Text,     nullable=False),   # SimSpec.model_dump_json()
    Column("diff_json",      Text,     nullable=True),    # list[SpecDiff] vs parent
```

```
    Column("change_reason",  Text,     nullable=True),
    Column("changed_by",     String,   nullable=True),    # user/system/calibration
    Column("created_at",     DateTime, server_default=func.now()),
    # named versions: "pre-reform", "post-reform", "client-briefing-march-15"
    Column("version_label",  String,   nullable=True),
)


# ── simulations ──────────────────────────────────────────────────────

simulations = Table(
    "simulations", metadata,
    Column("sim_id",       String,   primary_key=True),
    Column("spec_id",      String,   ForeignKey("sim_specs.spec_id"), nullable=False),
    Column("state",        String,   nullable=False, default="CREATED"),
    # runner_config: n_runs for ensemble, seed, parameter perturbations
    Column("runner_config_json", Text, nullable=True),
    Column("created_at",   DateTime, server_default=func.now()),
    Column("completed_at", DateTime, nullable=True),
    Column("error",        Text,     nullable=True),
)

Index("ix_simulations_state", simulations.c.state)
Index("ix_simulations_spec_id", simulations.c.spec_id)


# ── snapshots ────────────────────────────────────────────────────────

snapshots = Table(
    "snapshots", metadata,
    Column("snapshot_id",       String,   primary_key=True),
    Column("sim_id",            String,   ForeignKey("simulations.sim_id"), nullable=False),
    Column("tick",              Integer,  nullable=False),
    Column("label",             String,   nullable=False),
    Column("env_json",          Text,     nullable=False),
    Column("agent_states_json", Text,     nullable=False),
    Column("theory_states_json",Text,     nullable=False),
    Column("spec_id",           String,   nullable=True),  # SimSpec version at snapshot time
    Column("timestamp",         DateTime, server_default=func.now()),
)

Index("ix_snapshots_sim_id", snapshots.c.sim_id)
Index("ix_snapshots_label",  snapshots.c.sim_id, snapshots.c.label)


# ── metric_records ───────────────────────────────────────────────────
# High-volume time series. One row per (sim, tick, metric).
# Partition by sim_id in Postgres for production.

metric_records = Table(
    "metric_records", metadata,
    Column("id",        Integer, primary_key=True, autoincrement=True),
    Column("sim_id",    String,  ForeignKey("simulations.sim_id"), nullable=False),
    Column("tick",      Integer, nullable=False),
    Column("metric_id", String,  nullable=False),
    Column("env_key",   String,  nullable=False),
    Column("value",     Float,   nullable=False),
)

Index("ix_metric_records_sim_tick", metric_records.c.sim_id, metric_records.c.tick)


# ── narrative_entries ────────────────────────────────────────────────

narrative_entries = Table(
    "narrative_entries", metadata,
    Column("entry_id",     String,   primary_key=True),
    Column("sim_id",       String,   ForeignKey("simulations.sim_id"), nullable=False),
    Column("tick",         Integer,  nullable=False),
    Column("content",      Text,     nullable=False),  # plain English from Claude
```

```python
    Column("triggered_by", String,   nullable=True),   # "threshold_X" | "snapshot_Y"
    Column("created_at",   DateTime, server_default=func.now()),
)

Index("ix_narrative_sim_tick", narrative_entries.c.sim_id, narrative_entries.c.tick)


# ── forge_sessions ──────────────────────────────────────────────────────
# Completed forge sessions archived to Postgres. Active sessions live in Redis.

forge_sessions = Table(
    "forge_sessions", metadata,
    Column("session_id",      String,   primary_key=True),
    Column("state",           String,   nullable=False),
    Column("simspec_id",      String,   ForeignKey("sim_specs.spec_id"), nullable=True),
    Column("conversation_json", Text,   nullable=True),
    Column("created_at",      DateTime, server_default=func.now()),
    Column("completed_at",    DateTime, nullable=True),
)


# ── calibration_proposals ───────────────────────────────────────────────

calibration_proposals = Table(
    "calibration_proposals", metadata,
    Column("proposal_id",  String,   primary_key=True),
    Column("sim_id",       String,   ForeignKey("simulations.sim_id"), nullable=False),
    Column("source_id",    String,   nullable=True),    # ResearchSourceSpec.source_id
    Column("env_key",      String,   nullable=False),
    Column("old_value",    Float,    nullable=False),
    Column("new_estimate", Float,    nullable=False),
    Column("confidence",   Float,    nullable=False, default=0.5),
    Column("rationale",    Text,     nullable=False),
    Column("status",       String,   nullable=False, default="pending"),  # pending|approved|rejected
    Column("created_at",   DateTime, server_default=func.now()),
    Column("resolved_at",  DateTime, nullable=True),
    Column("resolved_by",  String,   nullable=True),
)

Index("ix_calibration_sim_status", calibration_proposals.c.sim_id, calibration_proposals.c.status)
```

## Sim lifecycle state machine

```
CREATED ──► CONFIGURING (Forge intake running)
         └──► CONFIGURED (SimSpec validated, not yet started)
                  └──► RUNNING (SimRunner active)
                           ├──► PAUSED (manual pause via API)
                           │        └──► RUNNING (resume)
                           └──► COMPLETED
                                    └──► ARCHIVED
```

```python
# api/db/models.py

from enum import Enum


class SimState(str, Enum):
    CREATED     = "CREATED"
    CONFIGURING = "CONFIGURING"   # Forge in progress
    CONFIGURED  = "CONFIGURED"    # SimSpec ready, run not started
    RUNNING     = "RUNNING"
    PAUSED      = "PAUSED"
```

```
    COMPLETED   = "COMPLETED"
    ARCHIVED    = "ARCHIVED"
```

### `api/db/repository.py` — Repository layer

```python
# api/db/repository.py
from __future__ import annotations

import json
import uuid
from typing import Any

from sqlalchemy.ext.asyncio import AsyncConnection

from api.db.schema import (
    simulations, snapshots, metric_records,
    narrative_entries, sim_specs, calibration_proposals,
)
from core.sim_runner import MetricRecord, SimSnapshot
from core.spec import SimSpec


class SimRepository:
    """All DB access for the simulations domain. Accepts an open AsyncConnection."""

    def __init__(self, conn: AsyncConnection) -> None:
        self.conn = conn

    async def create_sim(
        self,
        spec: SimSpec,
        runner_config: dict[str, Any] | None = None,
    ) -> str:
        """Persist spec + create simulation row. Returns sim_id."""
        # Insert spec (version 1, no parent)
        await self.conn.execute(sim_specs.insert().values(
            spec_id=spec.spec_id,
            name=spec.name,
            domain=spec.domain,
            version_number=1,
            parent_spec_id=None,
            spec_json=spec.model_dump_json(),
        ))

        sim_id = str(uuid.uuid4())
        await self.conn.execute(simulations.insert().values(
            sim_id=sim_id,
            spec_id=spec.spec_id,
            state="CONFIGURED",
            runner_config_json=json.dumps(runner_config or {}),
        ))
        await self.conn.commit()
        return sim_id

    async def update_sim_state(self, sim_id: str, state: str) -> None:
        from sqlalchemy import update
        await self.conn.execute(
            update(simulations)
            .where(simulations.c.sim_id == sim_id)
            .values(state=state)
        )
        await self.conn.commit()

    async def save_snapshot(self, sim_id: str, snap: SimSnapshot) -> str:
        snapshot_id = str(uuid.uuid4())
        await self.conn.execute(snapshots.insert().values(
```

```python
        snapshot_id=snapshot_id,
        sim_id=sim_id,
        tick=snap.tick,
        label=snap.label,
        env_json=json.dumps(snap.env),
        agent_states_json=json.dumps(snap.agent_states),
        theory_states_json=json.dumps(snap.theory_states),
    ))
    await self.conn.commit()
    return snapshot_id

async def bulk_save_metrics(
    self, sim_id: str, records: list[MetricRecord]
) -> None:
    """Batch insert — called every N ticks to reduce DB round-trips."""
    if not records:
        return
    rows = [
        {
            "sim_id":    sim_id,
            "tick":      r.tick,
            "metric_id": r.metric_id,
            "env_key":   r.env_key,
            "value":     r.value,
        }
        for r in records
    ]
    await self.conn.execute(metric_records.insert(), rows)
    await self.conn.commit()

async def save_narrative_entry(
    self,
    sim_id: str,
    tick: int,
    content: str,
    triggered_by: str | None,
) -> str:
    entry_id = str(uuid.uuid4())
    await self.conn.execute(narrative_entries.insert().values(
        entry_id=entry_id,
        sim_id=sim_id,
        tick=tick,
        content=content,
        triggered_by=triggered_by,
    ))
    await self.conn.commit()
    return entry_id

async def get_narrative(self, sim_id: str) -> list[dict]:
    from sqlalchemy import select
    result = await self.conn.execute(
        select(narrative_entries)
        .where(narrative_entries.c.sim_id == sim_id)
        .order_by(narrative_entries.c.tick)
    )
    return [dict(row) for row in result]

async def list_snapshots(self, sim_id: str) -> list[dict]:
    from sqlalchemy import select
    result = await self.conn.execute(
        select(snapshots.c.snapshot_id, snapshots.c.tick, snapshots.c.label, snapshots.c.timestamp)
        .where(snapshots.c.sim_id == sim_id)
        .order_by(snapshots.c.tick)
    )
    return [dict(row) for row in result]

async def get_metrics(
    self, sim_id: str, metric_id: str | None = None
) -> list[dict]:
    from sqlalchemy import select
```

```
        q = select(metric_records).where(metric_records.c.sim_id == sim_id)
        if metric_id:
            q = q.where(metric_records.c.metric_id == metric_id)
        result = await self.conn.execute(q.order_by(metric_records.c.tick))
        return [dict(row) for row in result]
```

### Session store — Redis

```
# api/session_store.py
from __future__ import annotations

import json
from typing import Any

import redis.asyncio as aioredis

from forge.session import ForgeSession


class RedisSessionStore:
    """
    Active ForgeSession lives in Redis (TTL 24h).
    On ForgeSession completion, archive conversation_json to Postgres.
    JSON serialization: ForgeSession.to_dict() / from_dict().
    """

    TTL_SECONDS = 86_400  # 24 hours

    def __init__(self, redis_url: str = "redis://localhost:6379") -> None:
        self._redis = aioredis.from_url(redis_url, decode_responses=True)

    def _key(self, session_id: str) -> str:
        return f"forge:session:{session_id}"

    async def save(self, session: ForgeSession) -> None:
        key = self._key(session.session_id)
        await self._redis.setex(key, self.TTL_SECONDS, _serialize_session(session))

    async def load(self, session_id: str) -> ForgeSession | None:
        key = self._key(session_id)
        data = await self._redis.get(key)
        if data is None:
            return None
        return _deserialize_session(data)

    async def delete(self, session_id: str) -> None:
        await self._redis.delete(self._key(session_id))

    async def touch(self, session_id: str) -> None:
        """Reset TTL on activity."""
        await self._redis.expire(self._key(session_id), self.TTL_SECONDS)


def _serialize_session(session: ForgeSession) -> str:
    """ForgeSession → JSON string. Handles dataclass + SimSpec."""
    ...  # model_dump on SimSpec, asdict on ForgeSession dataclass fields


def _deserialize_session(data: str) -> ForgeSession:
    """JSON string → ForgeSession. Reconstructs SimSpec via model_validate."""
    ...
```

### Key design decisions

| Decision | Rationale |
|---|---|
| SQLAlchemy Core (not ORM) | The metric_records table is high-volume append-only. ORM overhead unnecessary. |
| SQLite dev / Postgres prod | Single `DB_URL` env var swap. `aiosqlite` for dev, `asyncpg` for prod. |
| Redis TTL 24h for active sessions | ForgeSession can have 30+ KB of conversation history. Not worth Postgres writes per turn. Archive on COMPLETE. |
| `bulk_save_metrics` batch insert | 365 ticks × N metrics = thousands of rows. Batch every 10 ticks. |
| Snapshot stores full env + agent + theory JSON | Self-contained. Can replay or diff without re-querying other tables. |
| `spec_id` column on `snapshots` | Records which SimSpec version was active at snapshot time. Critical for version diff. |

---

## Gap 2: `core/ensemble.py` — EnsembleRunner

```python
# core/ensemble.py
from __future__ import annotations

import asyncio
import random
import statistics
import uuid
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field
from enum import Enum
from typing import Any, Callable

from core.sim_runner import MetricRecord, SimRunner, SimSnapshot
from core.spec import SimSpec


# ── Result types ──────────────────────────────────────────────────

@dataclass
class PercentileBand:
    metric_id: str
    env_key:   str
    tick:      int
    p10:       float
    p50:       float
    p90:       float
    mean:      float
    std:       float


@dataclass
class MetricDistribution:
    """Final-tick distribution for a single metric across all runs."""
    metric_id:  str
    env_key:    str
    # raw values from all runs at final tick
    values:     list[float]
    p10:        float
    p50:        float
    p90:        float
    mean:       float
```

```python
    std:        float
    # e.g. P(value > 0.7) = 0.73
    probability_above: dict[float, float] = field(default_factory=dict)


@dataclass
class SpecDiff:
    """One changed field between two SimSpec versions."""
    field_path: str
    old_value:  Any
    new_value:  Any


@dataclass
class ScenarioDiff:
    """Distribution-level diff between two EnsembleResults (scenario A vs B)."""
    metric_id:   str
    env_key:     str
    delta_mean:  float   # B.mean - A.mean
    delta_p50:   float
    delta_p90:   float
    # KL divergence or Wasserstein distance — quantifies how different the distributions are
    divergence:  float


@dataclass
class EnsembleResult:
    ensemble_id:    str
    sim_id:         str
    n_requested:    int
    n_completed:    int
    spec_id:        str
    # per-metric final distributions
    distributions:  list[MetricDistribution]
    # percentile bands over time: list[PercentileBand] per metric per tick
    # indexed: {metric_id: list[PercentileBand]}
    bands:          dict[str, list[PercentileBand]]
    # raw run results for downstream use
    run_seeds:      list[int]
    run_summaries:  list[dict[str, Any]]  # final env + metric summary per run


class EnsembleStatus(str, Enum):
    RUNNING   = "running"
    COMPLETED = "completed"
    CANCELLED = "cancelled"
    FAILED    = "failed"


@dataclass
class EnsembleJob:
    ensemble_id:  str
    sim_id:       str
    spec:         SimSpec
    n_runs:       int
    base_seed:    int
    status:       EnsembleStatus = EnsembleStatus.RUNNING
    completed:    int = 0
    # partial results accumulated per run
    _run_metric_histories: list[list[MetricRecord]] = field(default_factory=list)
    _run_final_envs:       list[dict[str, float]]   = field(default_factory=list)
    cancelled:    bool = False


# ── EnsembleRunner ──────────────────────────────────────────────────────

class EnsembleRunner:
    """
    Wraps SimRunner. Runs N instances in parallel using a thread pool.
    Each instance gets the same SimSpec + a unique random seed + optional
```

```
    parameter perturbations from UncertaintySpec.

    Progressive results: listeners receive partial results as runs complete.
    """

    DEFAULT_N    = 100
    MAX_WORKERS  = 8

    def __init__(self) -> None:
        self._jobs:      dict[str, EnsembleJob]         = {}
        self._executor   = ThreadPoolExecutor(max_workers=self.MAX_WORKERS)
        self._listeners: dict[str, list[Callable]]      = {}

    async def launch(
        self,
        sim_id:        str,
        spec:          SimSpec,
        n_runs:        int | None = None,
        base_seed:     int | None = None,
        perturb_sigma: float = 0.0,  # Gaussian noise on initial_environment values
    ) -> str:
        n = n_runs or self.DEFAULT_N
        seed = base_seed if base_seed is not None else random.randint(0, 999_999)
        ensemble_id = str(uuid.uuid4())

        job = EnsembleJob(
            ensemble_id=ensemble_id,
            sim_id=sim_id,
            spec=spec,
            n_runs=n,
            base_seed=seed,
        )
        self._jobs[ensemble_id] = job
        asyncio.create_task(self._execute(job, perturb_sigma))
        return ensemble_id

    async def _execute(self, job: EnsembleJob, perturb_sigma: float) -> None:
        loop = asyncio.get_event_loop()

        async def run_one(run_idx: int) -> None:
            if job.cancelled:
                return
            run_seed = job.base_seed + run_idx
            perturbed_spec = _perturb_spec(job.spec, run_seed, perturb_sigma)
            result = await loop.run_in_executor(
                self._executor,
                _run_single,
                perturbed_spec,
                run_seed,
            )
            job._run_metric_histories.append(result["metric_history"])
            job._run_final_envs.append(result["final_env"])
            job.completed += 1

            # notify partial results
            for cb in self._listeners.get(job.ensemble_id, []):
                try:
                    await cb("run_complete", run_idx, job.completed, job.n_runs, result)
                except Exception:
                    pass

        # Fan out all runs — asyncio.gather wraps executor futures
        await asyncio.gather(*[run_one(i) for i in range(job.n_runs)])

        job.status = EnsembleStatus.COMPLETED
        result_obj = _compute_result(job)
        for cb in self._listeners.get(job.ensemble_id, []):
            try:
                await cb("job_complete", None, job.n_runs, job.n_runs, result_obj)
            except Exception:
```

```python
            pass

    def get_status(self, ensemble_id: str) -> dict:
        job = self._jobs[ensemble_id]
        return {
            "ensemble_id": ensemble_id,
            "status":      job.status.value,
            "completed":   job.completed,
            "total":       job.n_runs,
        }

    def get_result(self, ensemble_id: str) -> EnsembleResult:
        job = self._jobs[ensemble_id]
        if job.status != EnsembleStatus.COMPLETED:
            raise RuntimeError("Ensemble not yet complete")
        return _compute_result(job)

    def cancel(self, ensemble_id: str) -> None:
        job = self._jobs.get(ensemble_id)
        if job:
            job.cancelled = True
            job.status = EnsembleStatus.CANCELLED

    def add_listener(self, ensemble_id: str, callback: Callable) -> None:
        self._listeners.setdefault(ensemble_id, []).append(callback)

    def remove_listener(self, ensemble_id: str, callback: Callable) -> None:
        lst = self._listeners.get(ensemble_id, [])
        if callback in lst:
            lst.remove(callback)

    @staticmethod
    def compare(a: EnsembleResult, b: EnsembleResult) -> list[ScenarioDiff]:
        """
        Diff two ensemble results metric by metric.
        Returns list[ScenarioDiff] quantifying distribution divergence.
        Metrics matched by env_key.
        """
        a_idx = {d.env_key: d for d in a.distributions}
        b_idx = {d.env_key: d for d in b.distributions}
        diffs: list[ScenarioDiff] = []
        for key in set(a_idx) & set(b_idx):
            da, db = a_idx[key], b_idx[key]
            diffs.append(ScenarioDiff(
                metric_id=da.metric_id,
                env_key=key,
                delta_mean=db.mean - da.mean,
                delta_p50=db.p50 - da.p50,
                delta_p90=db.p90 - da.p90,
                divergence=_wasserstein_1d(da.values, db.values),
            ))
        return sorted(diffs, key=lambda d: abs(d.delta_mean), reverse=True)


# ── Helpers ──────────────────────────────────────────────────────────────

def _run_single(spec: SimSpec, seed: int) -> dict:
    """Execute one SimRunner in the thread pool. Returns metric_history + final_env."""
    random.seed(seed)
    runner = SimRunner(spec)
    runner.setup()
    runner.run()
    return {
        "metric_history": runner.metric_history,
        "final_env":      runner.get_current_env(),
        "snapshots":      runner.snapshots,
        "seed":           seed,
    }
```

```python
def _perturb_spec(spec: SimSpec, seed: int, sigma: float) -> SimSpec:
    """
    Add Gaussian noise to initial_environment values for this run.
    Values still clamped to [0, 1] after perturbation.
    sigma=0.0 → deterministic (only seed differs for agent randomness).
    """
    if sigma == 0.0:
        return spec
    rng = random.Random(seed)
    perturbed_env = {
        k: max(0.0, min(1.0, v + rng.gauss(0, sigma)))
        for k, v in spec.initial_environment.items()
    }
    return spec.model_copy(update={"initial_environment": perturbed_env})


def _compute_result(job: EnsembleJob) -> EnsembleResult:
    """Aggregate run results into distributions and percentile bands."""
    metrics = job.spec.metrics

    # Collect final-tick values per metric
    distributions: list[MetricDistribution] = []
    for metric in metrics:
        values: list[float] = []
        for history in job._run_metric_histories:
            matching = [r.value for r in history if r.metric_id == metric.metric_id]
            if matching:
                values.append(matching[-1])  # final tick value
        if not values:
            continue
        sorted_v = sorted(values)
        n = len(sorted_v)
        dist = MetricDistribution(
            metric_id=metric.metric_id,
            env_key=metric.env_key,
            values=values,
            p10=sorted_v[max(0, int(n * 0.10) - 1)],
            p50=sorted_v[int(n * 0.50)],
            p90=sorted_v[min(n - 1, int(n * 0.90))],
            mean=statistics.mean(values),
            std=statistics.stdev(values) if len(values) > 1 else 0.0,
        )
        # Compute common threshold probabilities
        for threshold in [0.3, 0.5, 0.7, 0.8, 0.9]:
            dist.probability_above[threshold] = sum(1 for v in values if v > threshold) / n
        distributions.append(dist)

    # Percentile bands over time
    bands: dict[str, list[PercentileBand]] = {}
    all_ticks = sorted({r.tick for h in job._run_metric_histories for r in h})
    for metric in metrics:
        metric_bands: list[PercentileBand] = []
        for tick in all_ticks:
            tick_values = [
                r.value
                for h in job._run_metric_histories
                for r in h
                if r.tick == tick and r.metric_id == metric.metric_id
            ]
            if not tick_values:
                continue
            s = sorted(tick_values)
            n = len(s)
            metric_bands.append(PercentileBand(
                metric_id=metric.metric_id,
                env_key=metric.env_key,
                tick=tick,
                p10=s[max(0, int(n * 0.10) - 1)],
                p50=s[int(n * 0.50)],
                p90=s[min(n - 1, int(n * 0.90))],
```

```
                mean=statistics.mean(tick_values),
                std=statistics.stdev(tick_values) if len(tick_values) > 1 else 0.0,
            ))
        bands[metric.metric_id] = metric_bands

    return EnsembleResult(
        ensemble_id=job.ensemble_id,
        sim_id=job.sim_id,
        n_requested=job.n_runs,
        n_completed=job.completed,
        spec_id=job.spec.spec_id,
        distributions=distributions,
        bands=bands,
        run_seeds=[job.base_seed + i for i in range(job.completed)],
        run_summaries=[{"final_env": env} for env in job._run_final_envs],
    )


def _wasserstein_1d(a: list[float], b: list[float]) -> float:
    """1D Wasserstein distance (Earth Mover's Distance). O(n log n)."""
    if not a or not b:
        return 0.0
    sa, sb = sorted(a), sorted(b)
    # Interpolate to common length
    n = max(len(sa), len(sb))
    def interp(lst, n):
        return [lst[min(int(i * len(lst) / n), len(lst) - 1)] for i in range(n)]
    ia, ib = interp(sa, n), interp(sb, n)
    return sum(abs(x - y) for x, y in zip(ia, ib)) / n
```

### Key design decisions

| Decision | Rationale |
|---|---|
| `asyncio.gather` over `asyncio.to_thread` loop | All N runs launch concurrently, throttled by `ThreadPoolExecutor(max_workers=8)`. Avoids sequential overhead. |
| `perturb_sigma` default 0.0 | Seed-only variation by default (agent randomness). Caller opts into parameter perturbation explicitly. |
| Percentile bands computed in-process | 100 runs × 365 ticks is ~36K records. Fits in memory. Postgres write happens once on completion. |
| `compare()` as static method | No job dependency. Takes two `EnsembleResult` objects. Usable from tests and API equally. |
| Wasserstein distance for divergence | More interpretable than KL divergence for bounded distributions. Shows how far scenario B "moved" from A. |

---

## Gap 3: Output Layer

### `core/narrative.py` — NarrativeAgent

```
# core/narrative.py
from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Any, Callable, Coroutine
```

```python
from anthropic import Anthropic

from core.sim_runner import MetricRecord, SimSnapshot
from core.spec import SimSpec


@dataclass
class NarrativeEntry:
    entry_id:     str
    tick:         int
    content:      str     # plain English, ready for report paste
    triggered_by: str     # "threshold:{metric_name}" | "snapshot:{label}" | "final"
    created_at:   float = field(default_factory=time.time)


class NarrativeAgent:
    """
    Subscribes to SimRunner events (via callback injection).
    Calls Claude API when significant events occur.
    Generates plain-English commentary for the narrative feed.

    Usage (by SimOrchestrator):
        agent = NarrativeAgent(spec, client)
        runner.on_snapshot = agent.on_snapshot
        runner.on_metric_threshold = agent.on_metric_threshold
    """

    SYSTEM_PROMPT = """
You are a strategic analysis agent writing narrative commentary for a consulting simulation.
Write in clear, authoritative prose suitable for executive briefings.
Reference specific actors, theory dynamics, and tick numbers.
Never expose model internals (no "env keys", "alpha", "beta", "normalized values").
Use display values and unit labels when available.
Maximum 3 sentences per entry. Be specific and causal.
"""

    def __init__(
        self,
        spec: SimSpec,
        client: Anthropic | None = None,
        on_entry: Callable[[NarrativeEntry], Coroutine] | None = None,
    ) -> None:
        self.spec   = spec
        self.client = client or Anthropic()
        self.entries: list[NarrativeEntry] = []
        self._on_entry = on_entry  # async callback → persist to DB

    async def on_snapshot(self, snapshot: SimSnapshot) -> None:
        """Called by SimRunner when any snapshot trigger fires."""
        entry = await self._generate(
            tick=snapshot.tick,
            trigger=f"snapshot:{snapshot.label}",
            context={
                "event":     "snapshot",
                "label":     snapshot.label,
                "tick":      snapshot.tick,
                "env":       self.spec.display_env(snapshot.env),
            },
        )
        self.entries.append(entry)
        if self._on_entry:
            await self._on_entry(entry)

    async def on_metric_threshold(
        self,
        metric_name: str,
        env_key:     str,
        value:       float,
        tick:        int,
```

```python
    recent_env:  dict[str, float],
) -> None:
    """Called when a metric crosses its snapshot_threshold."""
    display = self.spec.display_env(recent_env)
    entry = await self._generate(
        tick=tick,
        trigger=f"threshold:{metric_name}",
        context={
            "event":       "threshold_crossed",
            "metric":      metric_name,
            "env_key":     env_key,
            "value":       display.get(env_key, {}).get("display", value),
            "unit":        display.get(env_key, {}).get("unit", ""),
            "tick":        tick,
            "environment": display,
        },
    )
    self.entries.append(entry)
    if self._on_entry:
        await self._on_entry(entry)

async def on_final(self, tick: int, final_env: dict[str, float]) -> None:
    """Called at simulation completion."""
    entry = await self._generate(
        tick=tick,
        trigger="final",
        context={
            "event":       "simulation_complete",
            "tick":        tick,
            "environment": self.spec.display_env(final_env),
            "metrics":     [m.name for m in self.spec.metrics],
        },
    )
    self.entries.append(entry)
    if self._on_entry:
        await self._on_entry(entry)

async def compare_snapshots(
    self,
    label_a: str,
    snap_a:  SimSnapshot,
    label_b: str,
    snap_b:  SimSnapshot,
) -> str:
    """
    Generate narrative diff between two named snapshots.
    Returns plain-English string. Not stored — caller decides.
    """
    env_a = self.spec.display_env(snap_a.env)
    env_b = self.spec.display_env(snap_b.env)
    changed = _env_diff(env_a, env_b, threshold=0.02)
    response = self.client.messages.create(
        model="claude-opus-4-5",
        max_tokens=400,
        system=self.SYSTEM_PROMPT,
        messages=[{
            "role": "user",
            "content": (
                f"Simulation: {self.spec.name}\n"
                f"Comparing snapshot '{label_a}' (tick {snap_a.tick}) "
                f"to '{label_b}' (tick {snap_b.tick}).\n"
                f"Significant changes:\n{_format_diff(changed)}\n"
                "Write a 2-3 sentence narrative explaining what changed and why."
            ),
        }],
    )
    return response.content[0].text

async def _generate(self, tick: int, trigger: str, context: dict) -> NarrativeEntry:
    import uuid
```

```python
        response = self.client.messages.create(
            model="claude-opus-4-5",
            max_tokens=200,
            system=self.SYSTEM_PROMPT,
            messages=[{
                "role": "user",
                "content": (
                    f"Simulation: {self.spec.name} | Domain: {self.spec.domain}\n"
                    f"Event context: {_format_context(context)}\n"
                    "Write a concise narrative entry (2-3 sentences)."
                ),
            }],
        )
        return NarrativeEntry(
            entry_id=str(uuid.uuid4()),
            tick=tick,
            content=response.content[0].text,
            triggered_by=trigger,
        )


def _env_diff(
    env_a: dict[str, Any],
    env_b: dict[str, Any],
    threshold: float = 0.02,
) -> list[dict]:
    """Return keys where normalized value changed by more than threshold."""
    changes = []
    for key in set(env_a) & set(env_b):
        na = env_a[key].get("normalized", 0)
        nb = env_b[key].get("normalized", 0)
        if abs(nb - na) >= threshold:
            changes.append({
                "key":    key,
                "name":   env_b[key].get("display_name", key),
                "before": env_a[key].get("display", na),
                "after":  env_b[key].get("display", nb),
                "unit":   env_b[key].get("unit", ""),
                "delta":  nb - na,
            })
    return sorted(changes, key=lambda c: abs(c["delta"]), reverse=True)


def _format_diff(changes: list[dict]) -> str:
    lines = []
    for c in changes[:8]:  # top 8 changes to keep prompt bounded
        direction = "↑" if c["delta"] > 0 else "↓"
        lines.append(f"  {c['name']}: {c['before']}{c['unit']} → {c['after']}{c['unit']} {direction}")
    return "\n".join(lines)


def _format_context(ctx: dict) -> str:
    import json
    return json.dumps(ctx, indent=2, default=str)
```

### `core/reports.py` — Report export

```python
# core/reports.py
from __future__ import annotations

import json
from dataclasses import dataclass, field
from enum import Enum
from typing import Any
```

```python
class ReportFormat(str, Enum):
    JSON     = "json"
    MARKDOWN = "markdown"
    PDF      = "pdf"


@dataclass
class ReportSpec:
    """
    Declarative spec for what to include in a report.
    Passed to ReportRenderer to produce output.
    """
    sim_id:           str
    title:            str
    author:           str = ""
    date_range:       tuple[int, int] | None = None   # (tick_start, tick_end)
    snapshot_labels:  list[str] = field(default_factory=list)   # named snapshots to include
    metric_ids:       list[str] = field(default_factory=list)   # which metrics to chart
    include_narrative: bool = True
    include_ensemble:  bool = False
    ensemble_id:       str | None = None
    formats:          list[ReportFormat] = field(default_factory=lambda: [ReportFormat.MARKDOWN])


@dataclass
class GeneratedReport:
    report_id:  str
    sim_id:     str
    spec:       ReportSpec
    format:     ReportFormat
    content:    bytes   # raw file content
    filename:   str
    created_at: float


class ReportRenderer:
    """
    Assembles report data from DB and renders to requested format(s).

    Markdown template:
        # {title}
        **Simulation:** {spec.name}  |  **Generated:** {date}

        ## Executive Summary
        {narrative entries, chronological}

        ## Outcome Metrics
        {metric time series, as Markdown table or embedded chart spec}

        ## Snapshots
        {snapshot comparison tables}

        ## Ensemble Results (if requested)
        {p10/p50/p90 bands, scenario diffs}

    PDF: rendered from Markdown via markdown-pdf or weasyprint.
    Same md-to-pdf pattern as Hormuz export.
    """

    def render_markdown(
        self,
        spec:          ReportSpec,
        narrative:     list[dict],
        snapshots:     list[dict],
        metrics:       list[dict],
        ensemble:      Any | None = None,
    ) -> bytes:
        """Assemble Markdown string and return UTF-8 encoded bytes."""
        lines: list[str] = []
        lines.append(f"# {spec.title}\n")
```

```python
        if spec.author:
            lines.append(f"**Author:** {spec.author}  \n")

        if spec.include_narrative and narrative:
            lines.append("\n## Narrative Feed\n")
            for entry in narrative:
                lines.append(f"**Tick {entry['tick']}** ({entry['triggered_by']})")
                lines.append(f"\n{entry['content']}\n")

        if snapshots:
            lines.append("\n## Snapshots\n")
            for snap in snapshots:
                lines.append(f"### {snap['label']} (tick {snap['tick']})\n")
                # env table — top 10 keys by value
                env = json.loads(snap["env_json"])
                top = sorted(env.items(), key=lambda x: x[1], reverse=True)[:10]
                lines.append("| Variable | Value |")
                lines.append("|----------|-------|")
                for k, v in top:
                    lines.append(f"| {k} | {v:.3f} |")
                lines.append("")

        if spec.include_ensemble and ensemble:
            lines.append("\n## Ensemble Results\n")
            for dist in ensemble.distributions:
                lines.append(
                    f"**{dist.env_key}**: "
                    f"p10={dist.p10:.3f} / p50={dist.p50:.3f} / p90={dist.p90:.3f} "
                    f"(mean={dist.mean:.3f}, N={len(dist.values)})\n"
                )

        return "\n".join(lines).encode("utf-8")

    def render_pdf(self, markdown_content: bytes) -> bytes:
        """
        Convert Markdown → PDF.
        Uses weasyprint (same dependency chain as Hormuz _writer.py pattern).
        markdown_content: UTF-8 bytes from render_markdown().
        Returns PDF bytes.
        """
        import markdown
        from weasyprint import HTML, CSS
        html_body = markdown.markdown(
            markdown_content.decode("utf-8"),
            extensions=["tables", "fenced_code"],
        )
        html = f"""<!DOCTYPE html>
<html><head><meta charset="utf-8">
<style>
  body {{ font-family: Georgia, serif; max-width: 800px; margin: 40px auto; }}
  h1 {{ border-bottom: 2px solid #333; }}
  table {{ border-collapse: collapse; width: 100%; }}
  td, th {{ border: 1px solid #ccc; padding: 6px 10px; }}
</style>
</head><body>{html_body}</body></html>"""
        return HTML(string=html).write_pdf()

    def render_json(
        self,
        spec:      ReportSpec,
        narrative: list[dict],
        snapshots: list[dict],
        metrics:   list[dict],
        ensemble:  Any | None = None,
    ) -> bytes:
        """Structured JSON for Portal consumption."""
        payload = {
            "report_spec": {
                "sim_id": spec.sim_id,
                "title":  spec.title,
```

```
                "author": spec.author,
            },
            "narrative":  narrative,
            "snapshots":  [
                {"label": s["label"], "tick": s["tick"], "env": json.loads(s["env_json"])}
                for s in snapshots
            ],
            "metrics":    metrics,
        }
        if ensemble:
            payload["ensemble"] = {
                "distributions": [
                    {
                        "metric_id": d.metric_id,
                        "env_key":   d.env_key,
                        "p10": d.p10, "p50": d.p50, "p90": d.p90,
                        "mean": d.mean, "std": d.std,
                        "probability_above": d.probability_above,
                    }
                    for d in ensemble.distributions
                ],
            }
        return json.dumps(payload, indent=2, default=str).encode("utf-8")
```

### Key design decisions

| Decision | Rationale |
|---|---|
| `NarrativeAgent` injected as callback, not subclass | SimRunner stays unchanged. NarrativeAgent is an optional side-channel. |
| `on_entry` async callback pattern | Lets SimOrchestrator persist entries to Postgres in real time without blocking the tick loop. |
| `compare_snapshots()` returns string, not stored | Caller (API endpoint) decides whether to persist. Keeps the method reusable. |
| `ReportSpec` as declarative data model | Portal can construct and store report specs. Rendering is deterministic from spec + DB data. |
| PDF via weasyprint + Markdown intermediate | Matches the `_writer.py` pattern already proven in Hormuz. No new toolchain. |
| `render_json()` for Portal | Portal consumes structured JSON, renders its own charts. Markdown/PDF for human-readable deliverables. |

---

## SimSpec Versioning

```
# core/versioning.py
from __future__ import annotations

import json
import uuid
from dataclasses import dataclass
from typing import Any

from core.spec import SimSpec


@dataclass
```

```python
class SimSpecVersion:
    version_id:       str
    spec_id:          str
    version_number:   int
    parent_version_id: str | None
    diff_json:        list[SpecDiff]  # what changed from parent
    change_reason:    str
    changed_by:       str            # "user:{name}" | "calibration" | "system"
    version_label:    str | None     # "pre-reform", "post-reform"
    created_at:       float


def diff_simspecs(v1: SimSpec, v2: SimSpec) -> list["SpecDiff"]:
    """
    Compare two SimSpec instances field by field.
    Returns list[SpecDiff] — changes from v1 to v2.

    Checks:
    - initial_environment: per-key value changes
    - actors: added/removed, changed beliefs/desires/capabilities
    - theories: added/removed, changed parameters
    - timeframe, uncertainty: field-level diffs
    - env_key_specs: added/removed annotations
    """
    from dataclasses import dataclass

    diffs: list[SpecDiff] = []

    # Environment diffs
    env1, env2 = v1.initial_environment, v2.initial_environment
    for key in set(env1) | set(env2):
        old = env1.get(key)
        new = env2.get(key)
        if old != new:
            diffs.append(SpecDiff(
                field_path=f"initial_environment.{key}",
                old_value=old,
                new_value=new,
            ))

    # Theory parameter diffs
    t1 = {t.theory_id: t for t in v1.theories}
    t2 = {t.theory_id: t for t in v2.theories}
    for tid in set(t1) | set(t2):
        if tid not in t1:
            diffs.append(SpecDiff(f"theories.{tid}", None, t2[tid].model_dump()))
        elif tid not in t2:
            diffs.append(SpecDiff(f"theories.{tid}", t1[tid].model_dump(), None))
        else:
            for param, val in t2[tid].parameters.items():
                old_val = t1[tid].parameters.get(param)
                if old_val != val:
                    diffs.append(SpecDiff(
                        f"theories.{tid}.parameters.{param}", old_val, val
                    ))

    # Timeframe diffs
    for field in ("total_ticks", "tick_unit", "start_date"):
        old = getattr(v1.timeframe, field)
        new = getattr(v2.timeframe, field)
        if old != new:
            diffs.append(SpecDiff(f"timeframe.{field}", old, new))

    return diffs


def branch_simspec(
    base: SimSpec,
    branch_name: str,
    change_reason: str = "",
```

```
) -> SimSpec:
    """
    Create a new SimSpec branched from base.
    New spec_id, same structure, parent_spec_id recorded in DB (not on SimSpec itself).
    """
    new_spec = base.model_copy(update={
        "spec_id":     str(uuid.uuid4()),
        "name":        f"{base.name} — {branch_name}",
        "description": f"Branch of '{base.name}': {change_reason}",
    })
    return new_spec
```

---

## Data Feed Agent

```
# core/calibration.py
from __future__ import annotations

import time
import uuid
from dataclasses import dataclass, field
from typing import Any

from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.interval import IntervalTrigger


@dataclass
class CalibrationProposal:
    """
    Created by DataFeedAgent when drift detected.
    Consultant approves or rejects via API.
    On approval: new SimSpec version created, parameter updated, justification recorded.
    """
    proposal_id:  str = field(default_factory=lambda: str(uuid.uuid4()))
    sim_id:       str = ""
    source_id:    str = ""        # ResearchSourceSpec.source_id that triggered this
    env_key:      str = ""
    old_value:    float = 0.0
    new_estimate: float = 0.0
    confidence:   float = 0.5    # 0–1: how confident the data source is
    rationale:    str = ""       # plain-English explanation for the consultant
    status:       str = "pending"  # "pending" | "approved" | "rejected"
    created_at:   float = field(default_factory=time.time)


DRIFT_THRESHOLD = 0.05  # normalized units — proposals only created above this


class DataFeedAgent:
    """
    Scheduled recalibration agent. One job per running simulation.
    Pattern: APScheduler background scheduler (same as Hormuz lifespan pattern).

    Every 6 hours per simulation:
    1. Re-query each ResearchSourceSpec attached to the SimSpec.
    2. Run extraction pass (same as ScopingAgent._extraction_pass()).
    3. Compare extracted parameter_hints to current spec values.
    4. If drift > DRIFT_THRESHOLD: create CalibrationProposal.
    5. Proposals queued in DB for consultant review.
    """

    INTERVAL_HOURS = 6
```

```python
    def __init__(self) -> None:
        self._scheduler = BackgroundScheduler()
        self._active_jobs: dict[str, str] = {}  # sim_id → APScheduler job_id

    def start(self) -> None:
        self._scheduler.start()

    def shutdown(self) -> None:
        self._scheduler.shutdown(wait=False)

    def register_simulation(self, sim_id: str, spec_json: str) -> None:
        """Add a cron job for this simulation. Called when sim starts RUNNING."""
        job = self._scheduler.add_job(
            func=_calibration_job,
            trigger=IntervalTrigger(hours=self.INTERVAL_HOURS),
            args=[sim_id, spec_json],
            id=f"calibration_{sim_id}",
            replace_existing=True,
        )
        self._active_jobs[sim_id] = job.id

    def deregister_simulation(self, sim_id: str) -> None:
        """Remove job when simulation completes or is archived."""
        job_id = self._active_jobs.pop(sim_id, None)
        if job_id:
            try:
                self._scheduler.remove_job(job_id)
            except Exception:
                pass


def _calibration_job(sim_id: str, spec_json: str) -> None:
    """
    Synchronous function executed by APScheduler in background thread.
    1. Deserialize SimSpec from spec_json.
    2. Re-run research queries from spec.research_sources.
    3. Run extraction pass via Claude API.
    4. For each env key with drift > DRIFT_THRESHOLD: persist CalibrationProposal.
    """
    import asyncio
    asyncio.run(_async_calibration(sim_id, spec_json))


async def _async_calibration(sim_id: str, spec_json: str) -> None:
    from anthropic import Anthropic
    from core.spec import SimSpec
    from forge.researchers import (
        ArxivResearcher, FredResearcher, WorldBankResearcher, NewsResearcher,
    )

    spec = SimSpec.model_validate_json(spec_json)
    client = Anthropic()

    # Re-query sources defined on the spec
    research_tasks = []
    for source in spec.research_sources:
        if source.source_type == "arxiv":
            research_tasks.append(ArxivResearcher().search(source.query))
        elif source.source_type == "fred":
            research_tasks.append(FredResearcher().fetch([source.query]))
        elif source.source_type == "world_bank":
            research_tasks.append(WorldBankResearcher().fetch([source.query], []))
        elif source.source_type == "news":
            research_tasks.append(NewsResearcher().search(source.query, days_back=30))

    import asyncio
    results = await asyncio.gather(*research_tasks, return_exceptions=True)

    # Extract parameter hints via Claude (same prompt as ScopingAgent._extraction_pass)
```

```
    # Compare hints to spec.initial_environment
    # Persist CalibrationProposal rows for drifted keys
    # (implementation calls SimRepository via DB connection)
    ...


# Singleton — initialized in FastAPI lifespan
data_feed_agent = DataFeedAgent()
```

---

## `api/main.py` — Application lifespan

```
# api/main.py
from __future__ import annotations

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from api.routers import simulations, ensembles, narrative, reports, calibration, versioning
from api.routers.forge import router as forge_router
from api.ws.handlers import router as ws_router
from core.calibration import data_feed_agent
from core.ensemble import EnsembleRunner

logger = logging.getLogger(__name__)


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Initialize DB (create tables if missing)
    from api.db.connection import init_db
    await init_db()
    logger.info("Database initialized")

    # Start calibration scheduler
    data_feed_agent.start()
    logger.info("DataFeedAgent scheduler started")

    # Expose EnsembleRunner singleton on app state
    app.state.ensemble_runner = EnsembleRunner()

    yield

    data_feed_agent.shutdown()
    logger.info("DataFeedAgent scheduler stopped")


app = FastAPI(
    title="Crucible API",
    version="0.3.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```python
# REST routers
app.include_router(forge_router)
app.include_router(simulations.router)
app.include_router(ensembles.router)
app.include_router(narrative.router)
app.include_router(reports.router)
app.include_router(calibration.router)
app.include_router(versioning.router)

# WebSocket
app.include_router(ws_router)


@app.get("/api/health")
async def health():
    return {"status": "ok", "version": "0.3.0"}
```

---

## API Endpoints

**api/routers/simulations.py**

```python
# api/routers/simulations.py
from __future__ import annotations

from fastapi import APIRouter, HTTPException, WebSocket, WebSocketDisconnect, Request
from pydantic import BaseModel, Field
from typing import Any

router = APIRouter(prefix="/api/simulations", tags=["simulations"])


# ── Request / Response models ──────────────────────────────────────────

class SimCreateRequest(BaseModel):
    spec_id:  str | None = None  # use existing spec
    spec:     dict | None = None  # inline SimSpec dict (for testing)
    runner_config: dict[str, Any] = Field(default_factory=dict)


class SimCreateResponse(BaseModel):
    sim_id:  str
    spec_id: str
    state:   str


class SnapshotRequest(BaseModel):
    label: str


class ShockRequest(BaseModel):
    env_key: str
    delta:   float  # signed, applied as normalized delta before clamping


class SimStatusResponse(BaseModel):
    sim_id:      str
    state:       str
    tick:        int | None
    spec_id:     str
    created_at:  str
```

```python
    completed_at: str | None


# ── Endpoints ──────────────────────────────────────────────────────────

@router.post("", response_model=SimCreateResponse)
async def create_simulation(req: SimCreateRequest, request: Request):
    """Create simulation from SimSpec. State transitions to CONFIGURED."""
    ...


@router.get("/{sim_id}", response_model=SimStatusResponse)
async def get_simulation(sim_id: str):
    """Current state, tick, progress."""
    ...


@router.post("/{sim_id}/start")
async def start_simulation(sim_id: str, request: Request):
    """
    Start the SimRunner. State: CONFIGURED → RUNNING.
    Registers simulation with DataFeedAgent.
    Launches asyncio task: setup() + run_async().
    """
    ...


@router.post("/{sim_id}/pause")
async def pause_simulation(sim_id: str):
    """Signal SimRunner to pause after current tick."""
    ...


@router.post("/{sim_id}/resume")
async def resume_simulation(sim_id: str):
    """Resume from PAUSED."""
    ...


@router.get("/{sim_id}/snapshots")
async def list_snapshots(sim_id: str) -> list[dict]:
    """List all named snapshots for this simulation."""
    ...


@router.get("/{sim_id}/snapshots/{label}")
async def get_snapshot(sim_id: str, label: str) -> dict:
    """Retrieve a named snapshot by label."""
    ...


@router.post("/{sim_id}/snapshots")
async def take_snapshot(sim_id: str, req: SnapshotRequest) -> dict:
    """Manually trigger a named snapshot now."""
    ...


@router.get("/{sim_id}/metrics")
async def get_metrics(sim_id: str, metric_id: str | None = None) -> list[dict]:
    """
    Time series for all metrics (or one metric).
    Returns: [{tick, metric_id, env_key, value, display_value, unit}]
    """
    ...


@router.post("/{sim_id}/shocks")
async def inject_shock(sim_id: str, req: ShockRequest) -> dict:
    """Inject a runtime shock. Applied at next tick boundary."""
    ...
```

```python
@router.websocket("/{sim_id}/stream")
async def sim_stream(websocket: WebSocket, sim_id: str):
    """
    Live tick-by-tick stream.
    Messages: {"type": "tick", "tick": N, "env": {...}, "metrics": [...]}
              {"type": "snapshot", "label": "...", "tick": N}
              {"type": "narrative", "entry": {...}}
              {"type": "completed", "tick": N}
    """
    await websocket.accept()
    # Subscribe to SimRunner event bus for this sim_id
    ...
```

**api/routers/ensembles.py**

```python
# api/routers/ensembles.py
from __future__ import annotations

from fastapi import APIRouter, Request, WebSocket
from pydantic import BaseModel, Field

router = APIRouter(prefix="/api", tags=["ensemble"])


class EnsembleRequest(BaseModel):
    n_runs:        int   = Field(default=100, ge=1, le=1000)
    base_seed:     int | None = None
    perturb_sigma: float = Field(default=0.0, ge=0.0, le=0.2)


class EnsembleResponse(BaseModel):
    ensemble_id: str
    sim_id:      str
    n_runs:      int
    status:      str


class CompareRequest(BaseModel):
    ensemble_id_a: str
    ensemble_id_b: str


@router.post("/simulations/{sim_id}/ensemble", response_model=EnsembleResponse)
async def launch_ensemble(sim_id: str, req: EnsembleRequest, request: Request):
    """Launch N ensemble runs for this simulation's SimSpec."""
    ...


@router.get("/ensembles/{ensemble_id}")
async def get_ensemble_status(ensemble_id: str) -> dict:
    """Status + partial distribution results as runs complete."""
    ...


@router.get("/ensembles/{ensemble_id}/results")
async def get_ensemble_results(ensemble_id: str) -> dict:
    """Full EnsembleResult: distributions, percentile bands, run summaries."""
    ...


@router.post("/ensembles/compare")
async def compare_ensembles(req: CompareRequest) -> list[dict]:
    """Compare two ensemble results. Returns list[ScenarioDiff]."""
    ...
```

```python
@router.websocket("/ensembles/{ensemble_id}/stream")
async def ensemble_stream(websocket: WebSocket, ensemble_id: str):
    """
    Stream partial results as runs complete.
    Messages: {"type": "run_complete", "run_idx": N, "completed": N, "total": N}
              {"type": "job_complete", "results": {...}}
    """
    ...
```

**api/routers/narrative.py**

```python
# api/routers/narrative.py
from fastapi import APIRouter

router = APIRouter(prefix="/api/simulations", tags=["narrative"])


@router.get("/{sim_id}/narrative")
async def get_narrative(sim_id: str) -> list[dict]:
    """Full chronological narrative feed for this simulation."""
    ...


@router.get("/{sim_id}/narrative/{tick}")
async def get_narrative_at_tick(sim_id: str, tick: int) -> list[dict]:
    """Narrative entries at or near a specific tick."""
    ...


@router.post("/{sim_id}/narrative/compare")
async def compare_snapshots_narrative(
    sim_id: str,
    label_a: str,
    label_b: str,
) -> dict:
    """Generate narrative diff between two named snapshots."""
    ...
```

**api/routers/reports.py**

```python
# api/routers/reports.py
from __future__ import annotations

from fastapi import APIRouter
from fastapi.responses import Response
from pydantic import BaseModel

router = APIRouter(prefix="/api", tags=["reports"])


class ReportRequest(BaseModel):
    title:             str
    author:            str = ""
    snapshot_labels:   list[str] = []
    metric_ids:        list[str] = []
    include_narrative: bool = True
    include_ensemble:  bool = False
    ensemble_id:       str | None = None
    formats:           list[str] = ["markdown"]  # "json" | "markdown" | "pdf"
```

```python
@router.post("/simulations/{sim_id}/reports")
async def generate_report(sim_id: str, req: ReportRequest) -> dict:
    """
    Enqueue report generation. Returns report_id.
    Report is assembled from DB data — no live simulation access needed.
    """
    ...


@router.get("/reports/{report_id}")
async def download_report(report_id: str, format: str = "markdown") -> Response:
    """
    Download generated report.
    Content-Type: application/json | text/markdown | application/pdf
    """
    ...
```

**api/routers/calibration.py**

```python
# api/routers/calibration.py
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter(prefix="/api/simulations", tags=["calibration"])


class ProposalDecision(BaseModel):
    reason: str = ""


@router.get("/{sim_id}/calibration/proposals")
async def list_proposals(sim_id: str, status: str = "pending") -> list[dict]:
    """List CalibrationProposals for this simulation."""
    ...


@router.post("/{sim_id}/calibration/proposals/{proposal_id}/approve")
async def approve_proposal(
    sim_id: str,
    proposal_id: str,
    body: ProposalDecision,
) -> dict:
    """
    Approve a CalibrationProposal.
    Side effects:
    1. Create new SimSpec version with updated env key.
    2. Record change_reason from proposal rationale + approval note.
    3. If simulation is RUNNING: schedule env update at next tick boundary.
    4. Mark proposal status = "approved".
    """
    ...


@router.post("/{sim_id}/calibration/proposals/{proposal_id}/reject")
async def reject_proposal(
    sim_id: str,
    proposal_id: str,
    body: ProposalDecision,
) -> dict:
    """Mark proposal rejected. No spec mutation."""
    ...
```

**api/routers/versioning.py**

```python
# api/routers/versioning.py
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter(prefix="/api/simspecs", tags=["versioning"])


class BranchRequest(BaseModel):
    branch_name:   str
    change_reason: str = ""


@router.get("/{spec_id}/versions")
async def list_versions(spec_id: str) -> list[dict]:
    """
    Full version history for a SimSpec.
    Includes version_number, version_label, change_reason, changed_by, created_at.
    """
    ...


@router.get("/{spec_id}/versions/{v1}/diff/{v2}")
async def diff_versions(spec_id: str, v1: str, v2: str) -> list[dict]:
    """
    Diff two SimSpec versions. v1, v2 are spec_ids (version rows have unique spec_ids).
    Returns list[SpecDiff]: field_path, old_value, new_value.
    """
    ...


@router.post("/{spec_id}/branch")
async def branch_spec(spec_id: str, req: BranchRequest) -> dict:
    """
    Create a scenario variant from this spec version.
    Returns new spec_id. Parent lineage recorded in DB.
    """
    ...
```

---

## Module Dependency Graph (full stack)

```
core/spec.py                  (no internal deps)
    ↑
core/agents/base.py           (no internal deps)
    ↑
core/theories/base.py         (BDIAgent type hint)
    ↑
core/theories/__init__.py     (registry)
    ↑
core/sim_runner.py            (spec, agents, theories)
    ↑                 ↑
core/ensemble.py          core/narrative.py     core/reports.py
(wraps SimRunner)         (Claude API calls)    (ReportRenderer)
    ↑                           ↑
core/versioning.py              |
```

```
(diff_simspecs, branch)       │
    ↑                         │
core/calibration.py ──────────┘
(APScheduler + DataFeedAgent)
    ↑
api/db/schema.py              (SQLAlchemy tables — no core deps)
    ↑
api/db/repository.py          (SimRepository)
    ↑
api/session_store.py          (Redis — ForgeSession only)
    ↑
api/routers/                  (FastAPI endpoints)
    ↑
forge/                        (ForgeSession, ScopingAgent, researchers)
```

No circular imports. `api/` depends on `core/` and `forge/`. `core/` has no `api/` or `forge/` imports.

---

## Environment variable reference

| Variable | Required | Description |
|---|---|---|
| `DATABASE_URL` | Yes | `sqlite+aiosqlite:///./crucible.db` (dev) or `postgresql+asyncpg://...` (prod) |
| `REDIS_URL` | Yes | `redis://localhost:6379` |
| `ANTHROPIC_API_KEY` | Yes | Claude API — NarrativeAgent, DataFeedAgent extraction pass |
| `FRED_API_KEY` | Yes | FRED economic data for calibration |
| `NEWS_API_KEY` | Yes | NewsAPI for calibration research |
| `MAX_ENSEMBLE_WORKERS` | No | Default 8. Thread pool size for EnsembleRunner. |
| `MAX_CONCURRENT_ENSEMBLES` | No | Default 3. Cap on running ensemble jobs. |
| `CALIBRATION_DRIFT_THRESHOLD` | No | Default 0.05. Normalized units. |

---

## Decision tables

### Storage backend per data type

| Data type | Store | Rationale |
|---|---|---|
| Active ForgeSession | Redis TTL 24h | Frequent small writes per turn. No durability needed until COMPLETE. |
| Completed ForgeSession | Postgres `forge_sessions` | Audit trail. |

| | | |
|---|---|---|
| SimSpec + versions | Postgres `sim_specs` | Version DAG, diff queries, branch support. |
| SimRunner state (live) | In-process (SimRunner object) | Tick loop is synchronous. DB write per tick would be 10–100x slower. |
| Snapshots | Postgres `snapshots` | Named, queryable, cross-run comparison. |
| Metric time series | Postgres `metric_records` | High-volume but append-only. Batch insert every 10 ticks. |
| Narrative entries | Postgres `narrative_entries` | Low-volume. Written as generated. |
| CalibrationProposals | Postgres `calibration_proposals` | Consultant review workflow. |
| EnsembleResult (live) | In-process (EnsembleJob) | Aggregated once on COMPLETE, then persisted. |
| Generated reports | In-process bytes → response | Not stored by default. Caller streams PDF/MD. |

### Ensemble concurrency model

| Approach | Chosen | Rationale |
|---|---|---|
| `asyncio.gather + ThreadPoolExecutor` | Yes | SimRunner.run() is CPU-bound sync. Thread pool prevents GIL contention. |
| `ProcessPoolExecutor` | No | Process startup overhead per run, pickling complexity. |
| Sequential runs in one thread | No | 100 runs × 365 ticks would block for minutes. |
| Celery distributed queue | No | Overkill for consulting firm scale. Adds Redis queue + worker ops. |

### Report formats

| Format | Consumer | Render path |
|---|---|---|
| JSON | Portal (React charts) | `render_json()` — structured data |
| Markdown | Consultant review | `render_markdown()` — paste-ready |
| PDF | Client deliverable | `render_markdown()` → `render_pdf()` (weasyprint) |

---

Next: ARCHITECTURE-PORTAL.md — client-facing Portal layer: React component tree, chart components consuming EnsembleResult bands and narrative feed, snapshot comparison UI, report download flow, authentication and multi-tenancy model.

---

## Engineering Review — Corrections

> *Applied after eng-manager review, 2026-03-24. All changes agreed before implementation. Grouped by file. Implement these alongside the base design — they are not optional.*

---

#### `core/spec.py`

**Move `SpecDiff` here** (not in `core/ensemble.py` or `core/versioning.py`):

```
@dataclass
class SpecDiff:
    """One changed field between two SimSpec versions. Lives here so both
    ensemble.py and versioning.py can import without circular dependencies."""
    field_path: str
    old_value:  Any
    new_value:  Any
```

Both `ensemble.py` and `versioning.py` do `from core.spec import SpecDiff`.

---

#### `core/agents/base.py` and `core/sim_runner.py`

**Thread-safe RNG — `rng: random.Random` parameter:**

`BDIAgent.__init__` and `SimRunner.__init__` must accept an `rng` parameter:

```
# core/agents/base.py
class BDIAgent:
    def __init__(self, spec: ActorSpec, rng: random.Random | None = None) -> None:
        self.rng = rng or random.Random()
        # All internal randomness uses self.rng, never the global random module
```

```
# core/sim_runner.py
class SimRunner:
    def __init__(self, spec: SimSpec, rng: random.Random | None = None) -> None:
        self.rng = rng or random.Random()
        # Pass self.rng to each BDIAgent on setup()
```

**Why:** `random.seed(seed)` in a thread sets global state. With 8 concurrent threads in `EnsembleRunner`, threads stomp each other's seeds. Ensemble results are not reproducible. `random.Random(seed)` is a local instance — fully thread-safe.

**Impact on ARCHITECTURE.md:** Note that `BDIAgent.decide()` and any random sampling in theory `update()` must use the `rng` passed in, not `random.random()` globally.

---

#### `core/ensemble.py`

**1. Remove `random.seed(seed)` from `_run_single`:**

```
def _run_single(spec: SimSpec, seed: int) -> dict:
    # DO NOT call random.seed(seed) — global state, not thread-safe
    rng = random.Random(seed)       # local instance, thread-safe
```

```
    runner = SimRunner(spec, rng=rng)
    runner.setup()
    runner.run()
    return { ... }
```

**2. Cache `EnsembleResult` on `EnsembleJob` — no double computation:**

```
@dataclass
class EnsembleJob:
    ...
    failed:   int = 0                              # runs that raised exceptions
    _result:  EnsembleResult | None = None         # cached after first compute
```

In `_execute()`:

```
job.status = EnsembleStatus.COMPLETED
job._result = _compute_result(job)        # compute ONCE, store on job
for cb in self._listeners.get(job.ensemble_id, []):
    await cb("job_complete", None, job.n_runs, job.n_runs, job._result)
```

In `get_result()`:

```
def get_result(self, ensemble_id: str) -> EnsembleResult:
    job = self._jobs[ensemble_id]
    if job.status != EnsembleStatus.COMPLETED or job._result is None:
        raise RuntimeError("Ensemble not yet complete")
    return job._result    # serve from cache, never recompute
```

**3. Fix O($n^2$) percentile band loop — pre-build index:**

```
def _compute_result(job: EnsembleJob) -> EnsembleResult:
    # Build index once: O(total_records)
    # Structure: {(metric_id, tick): [values across all runs]}
    index: dict[tuple[str, int], list[float]] = {}
    for history in job._run_metric_histories:
        for r in history:
            key = (r.metric_id, r.tick)
            index.setdefault(key, []).append(r.value)

    all_ticks = sorted({tick for (_, tick) in index})

    # Percentile bands: O(metrics × ticks) — no inner scan
    bands: dict[str, list[PercentileBand]] = {}
    for metric in job.spec.metrics:
        metric_bands = []
        for tick in all_ticks:
            tick_values = index.get((metric.metric_id, tick), [])
            if not tick_values:
                continue
            s = sorted(tick_values)
            n = len(s)
            metric_bands.append(PercentileBand(
                metric_id=metric.metric_id,
                env_key=metric.env_key,
                tick=tick,
                p10=s[max(0, int(n * 0.10) - 1)],
                p50=s[int(n * 0.50)],
                p90=s[min(n - 1, int(n * 0.90))],
                mean=statistics.mean(tick_values),
```

```
                std=statistics.stdev(tick_values) if len(tick_values) > 1 else 0.0,
            ))
        bands[metric.metric_id] = metric_bands
    ...
```

**Why:** Original loop was O(runs × ticks² × metrics). With 100 runs × 365 ticks × 5 metrics = ~66M iterations. Index pre-build reduces to O(n) + O(metrics × ticks).

**4. `asyncio.gather` with `return_exceptions=True`:**

```
# In _execute():
async def run_one(run_idx: int) -> None:
    if job.cancelled:
        return
    try:
        run_seed = job.base_seed + run_idx
        perturbed_spec = _perturb_spec(job.spec, run_seed, perturb_sigma)
        result = await loop.run_in_executor(self._executor, _run_single, perturbed_spec, run_seed)
        job._run_metric_histories.append(result["metric_history"])
        job._run_final_envs.append(result["final_env"])
        job.completed += 1
        for cb in self._listeners.get(job.ensemble_id, []):
            try:
                await cb("run_complete", run_idx, job.completed, job.n_runs, result)
            except Exception:
                pass
    except Exception as exc:
        job.failed += 1
        logger.warning("Ensemble run %d failed: %r", run_idx, exc)

await asyncio.gather(*[run_one(i) for i in range(job.n_runs)])
# Note: exceptions are handled inside run_one — gather always completes
```

Add `n_failed` to `EnsembleResult`:

```
@dataclass
class EnsembleResult:
    ...
    n_failed: int = 0   # runs that raised — non-zero means partial results
```

**5. `asyncio.get_running_loop()` — not `get_event_loop()`:**

```
async def _execute(self, job: EnsembleJob, perturb_sigma: float) -> None:
    loop = asyncio.get_running_loop()   # not get_event_loop() (deprecated 3.10+)
```

**6. `EnsembleRunner` is an app-level singleton with job eviction:**

```
class EnsembleRunner:
    ...
    JOB_TTL_SECONDS = 3600  # evict jobs 1 hour after completion

    def evict(self, ensemble_id: str) -> None:
        """
        Evict raw run data from a completed job. Keeps status and cached result.
        Call after results have been served to the client.
        """
        job = self._jobs.get(ensemble_id)
        if job and job.status == EnsembleStatus.COMPLETED:
            job._run_metric_histories.clear()
```

```
            job._run_final_envs.clear()

    async def cleanup_stale_jobs(self) -> int:
        """
        Remove jobs older than JOB_TTL_SECONDS. Called by a background task
        in the FastAPI lifespan every 30 minutes.
        Returns count of evicted jobs.
        """
        now = asyncio.get_event_loop().time()
        to_remove = [
            eid for eid, job in self._jobs.items()
            if job.status in (EnsembleStatus.COMPLETED, EnsembleStatus.FAILED, EnsembleStatus.CANCELLED)
        ]
        for eid in to_remove:
            self._jobs.pop(eid, None)
            self._listeners.pop(eid, None)
        return len(to_remove)
```

Add to `api/main.py` lifespan:

```
# Background cleanup task — runs every 30 minutes
async def _cleanup_loop(runner: EnsembleRunner):
    while True:
        await asyncio.sleep(1800)
        n = await runner.cleanup_stale_jobs()
        if n:
            logger.info("EnsembleRunner evicted %d stale jobs", n)

# In lifespan, after creating ensemble_runner:
app.state.ensemble_runner = EnsembleRunner()
asyncio.create_task(_cleanup_loop(app.state.ensemble_runner))
```

---

### `api/session_store.py` — `ForgeSession` serialization contract

Replace the `...` stubs with explicit `to_dict` / `from_dict` on `ForgeSession`:

```
# forge/session.py — add these methods to ForgeSession dataclass

def to_dict(self) -> dict:
    """
    Serialize ForgeSession to a JSON-safe dict.
    Rules:
    - ForgeState enum → .value (str)
    - SimSpec (Pydantic) → .model_dump()
    - SpecGap, ResearchResult (dataclasses) → dataclasses.asdict()
    - ForgeMessage.content: str or list[dict] (Claude tool-use blocks)
      → stored as-is (already JSON-serializable)
    - ResearchContext → dataclasses.asdict()
    """
    import dataclasses
    return {
        "session_id":           self.session_id,
        "state":                self.state.value,
        "simspec":              self.simspec.model_dump() if self.simspec else None,
        "conversation_history": [
            {"role": m.role, "content": m.content}
            for m in self.conversation_history
        ],
        "gaps":                 [dataclasses.asdict(g) for g in self.gaps],
        "research_context":     dataclasses.asdict(self.research_context) if self.research_context else None,
        "created_at":           self.created_at,
        "updated_at":           self.updated_at,
```

```
    }

@classmethod
def from_dict(cls, data: dict) -> "ForgeSession":
    """
    Reconstruct ForgeSession from serialized dict.
    Rules:
    - state: ForgeState(data["state"])
    - simspec: SimSpec.model_validate(data["simspec"]) if present
    - conversation_history: reconstruct ForgeMessage list
    - gaps: reconstruct SpecGap list
    - research_context: reconstruct ResearchContext if present
    """
    from forge.session import ForgeState, ForgeMessage, SpecGap, ResearchContext
    return cls(
        session_id=data["session_id"],
        state=ForgeState(data["state"]),
        simspec=SimSpec.model_validate(data["simspec"]) if data.get("simspec") else None,
        conversation_history=[
            ForgeMessage(role=m["role"], content=m["content"])
            for m in data.get("conversation_history", [])
        ],
        gaps=[SpecGap(**g) for g in data.get("gaps", [])],
        research_context=(
            ResearchContext(**data["research_context"])
            if data.get("research_context") else None
        ),
        created_at=data.get("created_at", 0.0),
        updated_at=data.get("updated_at", 0.0),
    )
```

Update `_serialize_session` / `_deserialize_session` in `api/session_store.py`:

```
def _serialize_session(session: ForgeSession) -> str:
    return json.dumps(session.to_dict())

def _deserialize_session(data: str) -> ForgeSession:
    return ForgeSession.from_dict(json.loads(data))
```

---

### `api/db/repository.py` — Engine injection + caller-controlled transactions

**Change constructor:**

```
class SimRepository:
    """All DB access for the simulations domain. Accepts an AsyncEngine.
    Each write method acquires its own short-lived connection.
    For atomic multi-step operations, use:
        async with engine.begin() as conn:
            repo = SimRepository(conn)
            await repo.save_snapshot(...)
            await repo.update_sim_state(...)
    """

    def __init__(self, engine: AsyncEngine) -> None:
        self._engine = engine
```

**Remove `commit()` from all methods.** Each method acquires its own connection for single-step operations:

```python
async def save_snapshot(self, sim_id: str, snap: SimSnapshot) -> str:
    snapshot_id = str(uuid.uuid4())
    async with self._engine.begin() as conn:
        await conn.execute(snapshots.insert().values(...))
    return snapshot_id
```

**For atomic multi-step (used by SimOrchestrator):**

```python
# SimOrchestrator pattern — explicit transaction:
async with engine.begin() as conn:
    repo = SimRepository.__new__(SimRepository)
    repo._engine = None
    repo._conn = conn   # or add a SimRepository.from_conn(conn) classmethod
    await repo._save_snapshot_tx(conn, sim_id, snap)
    await repo._save_narrative_tx(conn, sim_id, tick, content, triggered_by)
    await repo._update_state_tx(conn, sim_id, "COMPLETED")
# commit happens automatically on __aexit__
```

Add `tick_start`, `tick_end`, `limit`, `offset` to `get_metrics`:

```python
async def get_metrics(
    self,
    sim_id:     str,
    metric_id:  str | None = None,
    tick_start: int | None = None,
    tick_end:   int | None = None,
    limit:      int = 1000,
    offset:     int = 0,
) -> list[dict]:
    async with self._engine.connect() as conn:
        q = select(metric_records).where(metric_records.c.sim_id == sim_id)
        if metric_id:
            q = q.where(metric_records.c.metric_id == metric_id)
        if tick_start is not None:
            q = q.where(metric_records.c.tick >= tick_start)
        if tick_end is not None:
            q = q.where(metric_records.c.tick <= tick_end)
        q = q.order_by(metric_records.c.tick).limit(limit).offset(offset)
        result = await conn.execute(q)
        return [dict(row) for row in result]
```

---

## `SimOrchestrator` — new component (add as new section before API Endpoints)

```python
# api/orchestrator.py

class SimOrchestrator:
    """
    Wires SimRunner + NarrativeAgent + SimRepository for a single simulation run.
    API endpoints call SimOrchestrator, not individual components.

    ┌──────────────────────────────────────────────┐
    │  SimOrchestrator                              │
    │                                               │
    │  spec ──► SimRunner ──► tick loop            │
    │               │                               │
    │               ├──► NarrativeAgent.on_snapshot()  │
    │               │    (Claude API → NarrativeEntry)  │
```

```
│                │          │                        │
│                │          └──► SimRepository       │
│                │               .save_narrative_entry │
│                │                                   │
│                └──► bulk_save_metrics (every 10t)  │
│                     SimRepository.save_snapshot    │
│                                                    │
│  on_complete: atomic txn:                          │
│    save_final_snapshot                             │
│    save_final_narrative                            │
│    update_sim_state → COMPLETED                    │
└────────────────────────────────────────────────────┘
"""

METRIC_BATCH_SIZE = 10  # flush metrics every N ticks

def __init__(
    self,
    engine:      AsyncEngine,
    anthropic:   AsyncAnthropic,
) -> None:
    self._engine    = engine
    self._anthropic = anthropic
    self._repo      = SimRepository(engine)

async def run_single(self, sim_id: str, spec: SimSpec) -> None:
    """
    Full single-run lifecycle:
    1. State: CONFIGURED → RUNNING
    2. Wire NarrativeAgent callbacks
    3. Run SimRunner (asyncio.to_thread)
    4. Flush remaining metrics
    5. Atomic: save final snapshot + narrative + state → COMPLETED
    On exception: state → FAILED, error persisted
    """
    await self._repo.update_sim_state(sim_id, "RUNNING")
    rng = random.Random()   # no seed for single run (user didn't request reproducibility)
    runner = SimRunner(spec, rng=rng)
    agent  = NarrativeAgent(
        spec=spec,
        client=self._anthropic,
        on_entry=lambda e: self._repo.save_narrative_entry(
            sim_id, e.tick, e.content, e.triggered_by
        ),
    )
    runner.on_snapshot = agent.on_snapshot
    runner.on_metric_threshold = agent.on_metric_threshold

    metric_buffer: list[MetricRecord] = []

    async def on_tick(tick: int, metrics: list[MetricRecord]) -> None:
        metric_buffer.extend(metrics)
        if len(metric_buffer) >= self.METRIC_BATCH_SIZE:
            await self._repo.bulk_save_metrics(sim_id, metric_buffer)
            metric_buffer.clear()

    runner.on_tick = on_tick

    try:
        await asyncio.to_thread(runner.run)
        # Flush remaining metrics
        if metric_buffer:
            await self._repo.bulk_save_metrics(sim_id, metric_buffer)
        # Final narrative
        await agent.on_final(runner.current_tick, runner.get_current_env())
        # Atomic completion
        async with self._engine.begin() as conn:
            await conn.execute(
                update(simulations).where(simulations.c.sim_id == sim_id)
                .values(state="COMPLETED", completed_at=func.now())
```

```
                )
        except Exception as exc:
            logger.exception("SimRunner failed for sim_id=%s", sim_id)
            async with self._engine.begin() as conn:
                await conn.execute(
                    update(simulations).where(simulations.c.sim_id == sim_id)
                    .values(state="FAILED", error=str(exc))
                )
            raise

    async def run_ensemble(
        self,
        sim_id:        str,
        spec:          SimSpec,
        ensemble_runner: EnsembleRunner,
        n_runs:        int = 100,
        base_seed:     int | None = None,
        perturb_sigma: float = 0.0,
    ) -> str:
        """
        Launch EnsembleRunner. Returns ensemble_id.
        Persists EnsembleResult to DB on completion (via listener callback).
        """
        ensemble_id = await ensemble_runner.launch(
            sim_id=sim_id,
            spec=spec,
            n_runs=n_runs,
            base_seed=base_seed,
            perturb_sigma=perturb_sigma,
        )
        # Persist result to DB on job_complete
        async def on_event(event_type, *args):
            if event_type == "job_complete":
                result: EnsembleResult = args[-1]
                await self._persist_ensemble_result(sim_id, ensemble_id, result)
        ensemble_runner.add_listener(ensemble_id, on_event)
        return ensemble_id

    async def _persist_ensemble_result(
        self, sim_id: str, ensemble_id: str, result: EnsembleResult
    ) -> None:
        """Persist EnsembleResult summary to simulations table."""
        import json
        async with self._engine.begin() as conn:
            await conn.execute(
                update(simulations).where(simulations.c.sim_id == sim_id)
                .values(
                    state="COMPLETED",
                    completed_at=func.now(),
                    runner_config_json=json.dumps({
                        "ensemble_id":  ensemble_id,
                        "n_completed":  result.n_completed,
                        "n_failed":     result.n_failed,
                    }),
                )
            )
```

---

### `core/narrative.py` — `AsyncAnthropic` everywhere

```
from anthropic import AsyncAnthropic   # not Anthropic

class NarrativeAgent:
    def __init__(
        self,
```

```
        spec:     SimSpec,
        client:   AsyncAnthropic | None = None,
        on_entry: Callable[[NarrativeEntry], Coroutine] | None = None,
    ) -> None:
        self.client = client or AsyncAnthropic()

    async def _generate(self, tick: int, trigger: str, context: dict) -> NarrativeEntry:
        response = await self.client.messages.create(   # await — non-blocking
            model="claude-opus-4-6",
            max_tokens=200,
            system=self.SYSTEM_PROMPT,
            messages=[...],
        )
        ...
```

**Also applies to `ScopingAgent` in `forge/scoping.py` — use `AsyncAnthropic` throughout.**

**Model version:** Update all references from `claude-opus-4-5` to `claude-opus-4-6` (current).

---

### `core/calibration.py` — `DataFeedAgent` event loop bridging

```
class DataFeedAgent:
    def __init__(self) -> None:
        self._scheduler = BackgroundScheduler()
        self._active_jobs: dict[str, str] = {}
        self._loop: asyncio.AbstractEventLoop | None = None  # captured at start()

    def start(self) -> None:
        self._loop = asyncio.get_event_loop()   # capture BEFORE scheduler starts
        self._scheduler.start()

    def register_simulation(self, sim_id: str, spec_json: str) -> None:
        job = self._scheduler.add_job(
            func=self._calibration_job,    # bound method — has access to self._loop
            trigger=IntervalTrigger(hours=self.INTERVAL_HOURS),
            args=[sim_id, spec_json],
            ...
        )

    def _calibration_job(self, sim_id: str, spec_json: str) -> None:
        """Runs in APScheduler thread. Bridges to main event loop."""
        if self._loop is None or not self._loop.is_running():
            logger.error("Main event loop not available for calibration job sim_id=%s", sim_id)
            return
        future = asyncio.run_coroutine_threadsafe(
            _async_calibration(sim_id, spec_json),
            self._loop,
        )
        try:
            future.result(timeout=300)   # 5 min max per calibration cycle
        except Exception:
            logger.exception("Calibration job failed for sim_id=%s", sim_id)
```

**Why:** `asyncio.run()` creates a new event loop in the thread. Asyncpg connections are bound to the loop they were created on. `run_coroutine_threadsafe` submits the coroutine to the existing main loop instead, keeping all DB/Claude operations on the same loop.

---

### `api/main.py` — CORS config from environment

```python
# api/config.py (new file)
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL:    str = "sqlite+aiosqlite:///./crucible.db"
    REDIS_URL:       str = "redis://localhost:6379"
    ANTHROPIC_API_KEY: str = ""
    CORS_ORIGINS:    list[str] = ["http://localhost:3000", "http://localhost:5173"]
    MAX_ENSEMBLE_WORKERS: int = 8
    # ... other vars from environment variable reference table

    class Config:
        env_file = ".env"

settings = Settings()
```

```python
# api/main.py
from api.config import settings

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,   # not ["*"]
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

---

### `core/reports.py` — Gap 5 in render_markdown

```python
def render_markdown(
    self,
    spec:      SimSpec,          # ADDED — needed for display_env()
    narrative: list[dict],
    snapshots: list[dict],
    metrics:   list[dict],
    ensemble:  Any | None = None,
) -> bytes:
    ...
    if snapshots:
        for snap in snapshots:
            env_raw = json.loads(snap["env_json"])
            env = spec.display_env(env_raw)   # denormalize — shows display values
            # Sort by normalized value, display the display value
            top = sorted(env.items(), key=lambda x: x[1].get("normalized", 0), reverse=True)[:10]
            lines.append("| Variable | Value | Unit |")
            lines.append("|----------|-------|------|")
            for k, entry in top:
                name = entry.get("display_name", k)
                val  = entry.get("display", entry.get("normalized", 0))
                unit = entry.get("unit", "")
                lines.append(f"| {name} | {val:.3f} | {unit} |")
```

**Why:** Reports showing `iranian_sanctions_severity: 0.742` to consulting clients undermine the entire EnvKeySpec investment. `display_env()` translates to `74.2 billion USD` or whatever scale was specified.

---

## Environment variable reference (updated)

Add `CORS_ORIGINS` to the table:

| Variable | Required | Default | Description |
|---|---|---|---|
| `DATABASE_URL` | Yes | `sqlite+aiosqlite:///./crucible.db` | DB connection string |
| `REDIS_URL` | Yes | `redis://localhost:6379` | Redis for ForgeSession |
| `ANTHROPIC_API_KEY` | Yes | — | Claude API — Narrative, DataFeedAgent, ScopingAgent |
| `FRED_API_KEY` | Yes | — | FRED economic data |
| `NEWS_API_KEY` | Yes | — | NewsAPI calibration research |
| `CORS_ORIGINS` | No | `["http://localhost:3000","http://localhost:5173"]` | Allowed origins. **Must be set to Portal domain before enterprise deploy.** |
| `MAX_ENSEMBLE_WORKERS` | No | `8` | ThreadPoolExecutor size |
| `MAX_CONCURRENT_ENSEMBLES` | No | `3` | Cap on running ensemble jobs |
| `CALIBRATION_DRIFT_THRESHOLD` | No | `0.05` | Normalized units |

---

## Summary of all corrections

| # | File | Fix | Issue |
|---|---|---|---|
| 1 | `core/spec.py` | Move `SpecDiff` here | DRY — was in ensemble.py |
| 2 | `core/agents/base.py` | `rng: random.Random` param | Thread-safe seeding |
| 3 | `core/sim_runner.py` | `rng: random.Random` param, pass to agents | Thread-safe seeding |
| 4 | `core/ensemble.py` | Remove `random.seed()`, use `random.Random(seed)` in `_run_single` | Thread-safe seeding |
| 5 | `core/ensemble.py` | Cache `_result` on `EnsembleJob`, compute once | Double compute |
| 6 | `core/ensemble.py` | Pre-build `{(metric_id, tick): [values]}` index | $O(n^2) \rightarrow O(n)$ bands |
| 7 | `core/ensemble.py` | `return_exceptions=True` + `job.failed` counter | Cascade failure on 1 bad run |
| 8 | `core/ensemble.py` | `asyncio.get_running_loop()` | Deprecated 3.10+ |
| 9 | `core/ensemble.py` | Singleton + `evict()` + 30min cleanup task | Memory leak |
| 10 | `core/narrative.py` | `AsyncAnthropic` + `await` | Sync client blocks event loop |

| | | | |
|---|---|---|---|
| 11 | `core/calibration.py` | `run_coroutine_threadsafe` with captured main loop | `asyncio.run()` creates orphan loop |
| 12 | `api/db/repository.py` | `AsyncEngine` injection, `commit()` removed, `get_metrics` pagination | Connection lifecycle + atomicity |
| 13 | `api/orchestrator.py` | New `SimOrchestrator` class | Missing wiring layer |
| 14 | `api/session_store.py` | `ForgeSession.to_dict()` / `from_dict()` fully specified | Serialization stubs |
| 15 | `api/main.py` | `settings.CORS_ORIGINS` from env | CORS wildcard |
| 16 | `core/reports.py` | `simspec: SimSpec` param, call `display_env()` | Gap 5 regression in reports |

```
---

### Critical Files for Implementation

- `/d/dev/crucible/core/spec.py` — Add `EnvKeySpec` to SimSpec and `display_env()` method; this is the root data contract that every other layer touches.
- `/d/dev/crucible/core/ensemble.py` — New file; implement `EnsembleRunner`, `EnsembleResult`, `MetricDistribution`, `PercentileBand`, and `ScenarioDiff`; this is the primary gap-2 deliverable.
- `/d/dev/crucible/api/db/schema.py` — New file; the six SQLAlchemy table definitions are the foundation of the entire persistence layer; nothing else in the API can be wired without them.
- `/d/dev/crucible/core/narrative.py` — New file; `NarrativeAgent` and its callback injection into SimRunner; this is the client-deliverable output layer that closes gap-3.
- `/d/dev/hormuz-sim-dashboard/api/services/mc_runner.py` — Reference implementation for `asyncio.gather` + `ThreadPoolExecutor` + listener pattern; exact pattern for `EnsembleRunner._execute()`.
```